

s3.03 — Versioned configuration distribution with staged rollout and rollback

Index

- Interview framing
- Requirements — IC3 baseline
- Capacity and consequence — IC3 baseline
- Architecture and flow — IC3 baseline
- API and data model — IC3 baseline
- Immutable publication and fenced activation — IC3 baseline
- Staged rollout, acknowledgment, and rollback — IC3 baseline
- Failure, operations, and security
- Alternatives and IC4 stretch
- Level signal — IC3 baseline vs IC4 stretch
- Interview pushback and validation
- Recall card
- References

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - IC4 stretch changes the architecture only when a stronger requirement earns the cost.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: [ranked bank](#).
 - JD fit and preparation addition
 - No first-hand interview attribution.
 - This is a proposed design, not a claim about OCI internals or a reported accepted answer.
- Why this is hard
 - Naive: overwrite one configuration file and tell every agent to reload it.
 - First break: a valid but dangerous configuration reaches the whole fleet, while a delayed publisher overwrites the rollback.
 - Second break: a random 1% canary misses the one critical cohort, and an agent offline through every change cannot tell whether its cached policy is still authoritative.
 - Minimum primitives
 - Immutable content-addressed artifacts.
 - Signed manifests binding tenant and generation.
 - Conditional activation with a publisher fence.
 - Staged rollout and observed application health.
- 🗣️: "Content versions identify bytes; activation generations order decisions, including rollback."
- Opening script
 - "I will separate storing a configuration from activating it on a target group."
 - "I will clarify freshness, offline behavior, and whether the configuration contains security-critical rules."
 - "Immutable artifacts and signed manifests let agents validate bytes before applying them."
 - "The deep dive is a fenced staged rollout with acknowledgments tied to the exact assignment."
 - "I will close with rollback safety, disconnected agents, and the first fan-out bottleneck."

Phase	Clock	Deliverable
Intro	00:00–05:00	Establish role context and confirm the prompt

Phase	Clock	Deliverable
Requirements	05:00–11:00	Scope and one correctness invariant
Capacity and API	11:00–17:00	A number that forces a choice; retry contract
Architecture	17:00–30:00	Draw ownership and trace one request
Deep dive	30:00–45:00	Concurrency proof and a crash boundary
Operations and alternatives	45:00–50:00	Degradation, recovery, and flip condition
Wrap-up	50:00–54:00	Four-sentence recap and unresolved assumption
Interviewer questions	54:00–60:00	Team ownership and production priorities

- Clock note
 - The design slice is 45 minutes within the 60-minute round.
 - If the interviewer gives 45 minutes total, use a 2-minute intro and compress the remaining phases.
 - The interviewer's clock wins.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
What is distributed?	Service configuration; secret references only	Keep plaintext secrets out of shared artifacts
How fast must it converge?	Healthy connected agents within 60 s	Push hints plus bounded polling
Offline behavior?	Last-known-good within a declared validity policy	Availability does not imply indefinite security freshness
Who activates?	One authority per target group	Use CAS and an orchestrator fence
What proves success?	Loaded assignment plus stable workload metrics	Downloads alone cannot pass rollout gates

- Functional scope
 - Validate and publish immutable configurations.
 - Activate by target group and deployment generation.
 - Stage canary and wider rollout rings.
 - Collect assignment-specific readiness and health.
 - Roll back by publishing a higher activation generation.
- Targets below are interview assumptions, not product SLAs.

Requirement	Target	Enforced by
Artifact fetch	p99 < 100 ms from regional cache	Content-addressed object storage and CDN
Healthy propagation	p99 < 60 s	10 s jittered polling plus change hints
Activation availability	99.9% regional target	Replicated group authority
Canary safety	1% then 10%, 50%, 100%	Persisted gates with 10 min healthy soak
Offline routine configuration	Maximum 24 h stale by declared policy	Persisted verified artifact and expiry checks
Agent load failure	0 partial applications	Validate then atomic local swap

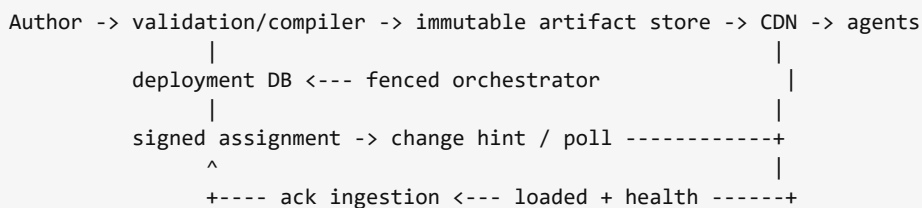
- Invariants
 - A signed manifest binds purpose, tenant, group, generation, schema, and content hash.
 - One target assignment at a generation names one exact artifact.
 - An older orchestrator cannot advance or roll back a newer deployment.
 - An acknowledgment only counts for the exact current target assignment.
 - Rollback increments activation generation even when it selects older content.
- Non-goals

- Using this channel as an unbounded secret store.
- Immediate global security revocation through an offline cache.
- Automatically proving that arbitrary configuration is semantically safe.

Capacity and consequence — IC3 baseline

- All workload values below are sizing assumptions.
- Fan-out
 - Assume 1M agents polling every 10 s: 100,000 assignment checks/s.
 - Decision: distribute signed assignment manifests through caches and jitter every poll.
- Artifact traffic
 - One 100 KB configuration sent to 1M agents is 100 GB of delivered bytes.
 - Decision: fetch immutable artifacts by hash through regional caches instead of through the metadata database.
- Acknowledgments
 - A full rollout produces at least 1M application acknowledgments.
 - Over 10 min, that is roughly 1,667/s before heartbeats.
 - Decision: partition acknowledgment ingestion and aggregate from deduplicated target state.
- History
 - 10,000 configurations with 100 versions at 100 KB is 100 GB of logical artifact data.
 - Decision: retain pinned rollback versions and garbage-collect only unreferenced artifacts after a safety window.

Architecture and flow — IC3 baseline



- Compiler
 - Buys deterministic schema validation and compatibility checks.
 - Pays schema evolution and validation cost.
 - Artifact store
 - Buys immutable verified bytes and cache reuse.
 - Pays retention and egress.
 - Deployment authority
 - Owns activation order and rollout gates.
 - Pays serialized changes per target group.
 - Agent
 - Owns validation and atomic local activation.
 - Pays disk persistence and application-specific health integration.
- Validate the authored configuration and compute its canonical content hash.
- Store immutable bytes and verify a read by hash.
- Claim deployment ownership and CAS the expected group generation.
- Distribute a signed assignment for the canary ring.
- Agent verifies scope, generation, signature, hash, and schema before atomic swap.
- Count current assignment acknowledgments and workload health before advancing the next ring.

API and data model — IC3 baseline

- Proposed API contracts.
 - Authenticate the tenant from credentials before looking up any resource.
 - Retain idempotency records for the documented retry horizon.

Surface	Contract	Idempotency and conflicts
POST /config-versions	Artifact schema and content; returns immutable version/hash	Tenant token deduplicates; validation errors are 422
POST /deployments	Group, artifactHash, expectedGeneration, rollout policy	409 on stale head or active conflicting deployment
POST /deployments/{id}:advance	Expected phase, generation, owner fence	409 if a newer owner or gate state won
POST /deployments/{id}:rollback	Previously approved artifact and expectedGeneration	Creates a higher generation; replay returns its deployment
GET /assignments/{target}	Signed current assignment and ETag	Target-authenticated read; stale response cannot lower applied generation
PUT /deployments/{id}/acks/{target}	Generation, assignmentHash, sequence, load state, health	Reject unassigned targets or mismatched generation/hash; stale sequence is no-op

State	Key	Fields and invariant
Artifact	(tenant, contentHash)	Canonical bytes, schema, creator, validation result
Group head	(tenant, groupId)	ActivationGeneration, deploymentId, ownerEpoch
Deployment	(tenant, deploymentId)	ArtifactHash, phase, expected targets, gate evidence
Assignment	(groupId, generation, targetId)	Exact manifest hash and requested artifact
Target observation	(deploymentId, targetId)	Assignment hash, observation sequence, loaded state, health
Outbox	(groupId, generation, phaseVersion)	Change notification intent committed with activation

- Reference DDL
 - Keys and uniqueness match the state table above and the [companion schema](#).
 - Artifact bytes and signed manifests live in content-addressed object storage behind the CDN; only their hashes and references are relational.

```

CREATE TABLE artifacts (
  tenant_id      text NOT NULL,
  content_hash   text NOT NULL,                -- sha256 of the canonical bytes; the identity
  schema_version text NOT NULL,
  content_ref    text NOT NULL,                -- object-storage key; bytes never live in this row
  size_bytes     integer NOT NULL,
  created_by     text NOT NULL,
  validation     text NOT NULL,
  create_token   text,                        -- POST /config-versions idempotency
  created_at     timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (tenant_id, content_hash),
  UNIQUE (tenant_id, create_token),
  CHECK (validation IN ('PASSED', 'FAILED')),
  CHECK (size_bytes > 0)
);

CREATE TABLE group_heads (
  tenant_id      text NOT NULL,
  group_id       text NOT NULL,
  activation_generation bigint NOT NULL DEFAULT 0, -- CAS target; rollback increments it too
  deployment_id  text,                          -- owner of the current generation
  owner_id       text,                          -- orchestrator holding the group lease
  owner_epoch    bigint NOT NULL DEFAULT 0,
  owner_lease_until timestampz,
  PRIMARY KEY (tenant_id, group_id),
  CHECK (activation_generation >= 0 AND owner_epoch >= 0)
);

CREATE TABLE deployments (
  tenant_id      text NOT NULL,
  deployment_id  text NOT NULL,
  group_id       text NOT NULL,
  generation     bigint NOT NULL,
  parent_generation bigint,
  artifact_hash  text NOT NULL,
  kind           text NOT NULL,
  phase         text NOT NULL DEFAULT 'CANARY',
  phase_version  bigint NOT NULL DEFAULT 0, -- expected value for :advance CAS
  rings         jsonb NOT NULL,           -- [1, 10, 50, 100] with 10 min soak
  inventory_version text NOT NULL,        -- versioned expected-target inventory; the gate
  denominator
  gate_evidence  jsonb,                    -- ack counts and metric snapshots per ring advance
  create_token   text,                    -- POST /deployments idempotency
  created_at     timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (tenant_id, deployment_id),
  UNIQUE (tenant_id, group_id, generation), -- one decision per generation
  UNIQUE (tenant_id, create_token),
  FOREIGN KEY (tenant_id, group_id) REFERENCES group_heads (tenant_id, group_id),
  FOREIGN KEY (tenant_id, artifact_hash) REFERENCES artifacts (tenant_id, content_hash),
  CHECK (kind IN ('ROLLOUT', 'ROLLBACK')),
  CHECK (phase IN ('CANARY', 'RING_10', 'RING_50', 'RING_100', 'COMPLETE', 'PAUSED', 'ABORTED')),
  CHECK (parent_generation IS NULL OR parent_generation < generation)
);

CREATE TABLE assignments (
  tenant_id      text NOT NULL,
  group_id       text NOT NULL,
  generation     bigint NOT NULL,
  target_id      text NOT NULL,
  deployment_id  text NOT NULL,
  ring           smallint NOT NULL,
  artifact_hash  text NOT NULL,
  manifest_hash  text NOT NULL,              -- hash of the signed manifest; acks must quote it exactly
  manifest_ref   text NOT NULL,              -- signed manifest in object storage
  PRIMARY KEY (tenant_id, group_id, generation, target_id), -- one exact artifact per target and
  generation
  FOREIGN KEY (tenant_id, deployment_id) REFERENCES deployments (tenant_id, deployment_id),
  FOREIGN KEY (tenant_id, artifact_hash) REFERENCES artifacts (tenant_id, content_hash),

```

```

CHECK (ring IN (1, 10, 50, 100))
);
CREATE INDEX assignments_by_target ON assignments (tenant_id, target_id, generation DESC); -- GET
/assignments/{target}

CREATE TABLE target_observations (
  tenant_id      text NOT NULL,
  deployment_id  text NOT NULL,
  target_id      text NOT NULL,
  generation     bigint NOT NULL,
  assignment_hash text NOT NULL,          -- must equal assignments.manifest_hash or the ack is
rejected
  observation_seq bigint NOT NULL,        -- monotonic per target; stale sequence is a no-op
  loaded_state   text NOT NULL,
  health         text NOT NULL DEFAULT 'UNKNOWN',
  observed_at    timestampz NOT NULL,
  PRIMARY KEY (tenant_id, deployment_id, target_id), -- one row per target; counts derive from rows
  FOREIGN KEY (tenant_id, deployment_id) REFERENCES deployments (tenant_id, deployment_id),
  CHECK (loaded_state IN ('DOWNLOADED', 'LOADED', 'REJECTED')),
  CHECK (health IN ('HEALTHY', 'UNHEALTHY', 'UNKNOWN')),
  CHECK (observation_seq >= 0)
);
CREATE INDEX observations_gate ON target_observations (tenant_id, deployment_id, loaded_state, health);

CREATE TABLE change_outbox (
  tenant_id      text NOT NULL,
  group_id       text NOT NULL,
  generation     bigint NOT NULL,
  phase_version  bigint NOT NULL,
  payload        jsonb NOT NULL,         -- change hint: group, generation, ring
  published_at   timestampz,
  PRIMARY KEY (tenant_id, group_id, generation, phase_version),
  FOREIGN KEY (tenant_id, group_id, generation) REFERENCES deployments (tenant_id, group_id, generation)
);
CREATE INDEX change_outbox_pending ON change_outbox (tenant_id, group_id) WHERE published_at IS NULL;

```

- Not captured by the DDL
 - Artifact bytes, signed manifests, and the CDN cache are object-storage state; signing keys are KMS handles referenced from the orchestrator, never columns.
 - The agent's verified last-known-good copy and its applied generation live on the target's disk.
 - `group_heads.deployment_id` carries no foreign key because `deployments` already references `group_heads`; the activation transaction writes both rows.
- Deviation from the companion
 - The companion keys content by `(config_id, config_version)` and rollouts by a stage list.
 - Here content is addressed by hash, `group_heads` holds the CAS generation and owner epoch, per-target `assignments` are explicit, and acknowledgments carry the assignment hash and observation sequence the endpoint table requires.

Immutable publication and fenced activation — IC3 baseline

- Content and activation are separate
 - Content hash identifies exact canonical bytes.
 - Activation generation identifies a decision for a target group.
 - Reusing old bytes for rollback still creates a new activation decision.
- Signed scope
 - Sign the entire canonical manifest with a domain separator.
 - Include tenant, group, target selector, generation, parent generation, schema version, content hash, and validity policy.
 - Pin trusted verification keys independently of the downloaded manifest.
 - Signing only the content hash leaves unsigned routing metadata open to substitution.

- Publication order
 - Persist and verify the immutable artifact first.
 - In one database transaction, verify expected group generation and owner epoch.
 - Advance the head and write the change outbox together.
 - A losing publisher leaves an unreferenced artifact that can be garbage-collected later.

```

ACTIVATE(group, expectedGeneration, ownerEpoch, artifactHash):
  require immutable artifact exists and passed validation
  transaction:
    lock group
    require currentGeneration == expectedGeneration
    require currentOwnerEpoch == ownerEpoch and owner lease is live
    create deployment at expectedGeneration + 1
    create exact ring assignments
    advance group head
    insert change outbox

```

- Agent acceptance
 - Verify signature and all scope fields before using the generation.
 - Reject lower generations.
 - Reject different content for an already accepted generation.
 - Persist verified bytes and manifest before reporting a successful application.

Staged rollout, acknowledgment, and rollback — IC3 baseline

- Rollout gates
 - Select a representative canary across platform and tenant cohorts.
 - A random 1% can miss the one high-volume or unusual client cohort.
 - Require exact assignment load acknowledgments and stable error/latency metrics.
 - Keep the denominator as a versioned inventory of expected targets.
 - Silent targets remain unknown until explicitly excluded with an owner and expiry.
- Acknowledgment semantics
 - DOWNLOADED proves bytes arrived.
 - LOADED proves the consumer accepted the configuration.
 - Healthy serving metrics prove the workload behaves acceptably.
 - Store a monotonic observation sequence separately from health.
 - Health can become unhealthy after a successful load.
 - Derive counts from target rows instead of incrementing counters on every retry.
- Atomic local application
 - Parse and validate a candidate without changing the active instance.
 - Apply through a versioned pointer or application-specific transaction.
 - On failure, keep the previous instance and report the rejected assignment.
 - A configuration requiring irreversible data migration cannot use a generic pointer rollback.
- Rollback
 - CAS the current generation and owner epoch.
 - Create a higher generation referencing the approved previous content.
 - Re-run compatibility and validity checks against the current binary and dependencies.
 - Keep deployment evidence so a retry cannot accidentally resume the aborted rollout.
- Disconnected agents
 - Load verified last-known-good from disk within its declared validity budget.
 - Recover a missed watch history through a complete signed snapshot.
 - Security policy expiry fails closed according to its profile rather than silently extending authority.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Bad config passes schema	Canary cohort	Workload errors after LOADED	Pause rollout; preserve previous assignment elsewhere	Higher-generation rollback
Old orchestrator resumes	One target group	Owner-epoch CAS failure	No activation change	Current owner continues
CDN serves stale manifest	Affected agents	Generation comparison	Keep current config	Poll authoritative head or refresh cache
Agent misses watch history	One target	Revision gap or compacted cursor	Keep valid current config	Fetch signed snapshot and resume
Control plane unavailable	New activations	API errors and freshness age	Routine agents retain valid cached config	Restore authority and reconcile target observations
Artifact store or CDN unavailable	New rollouts in the region	Fetch error rate by hash	Agents keep the verified cached config; an assignment whose bytes cannot be fetched is not applied	Restore the store; agents retry by immutable hash
Acknowledgment ingestion overloaded	Rollout progress visibility	Ack lag and dropped-write rate	Gates stall closed; no ring advances on missing evidence	Scale partitions; agents re-send the latest observation, deduplicated by target row
Rollback content incompatible with the running binary	One target group	Rollback validation failure before activation	Current generation stays active	Publish a corrected artifact as a higher generation; never skip validation for rollback
Signing key compromised	Every group trusting that key	Unexpected valid signatures; custody evidence	Agents keep the last verified config; pause activation	Revoke the trust key through the separate recovery channel and re-sign current heads

- Observe
 - Desired versus loaded generation by cohort.
 - Health regression after activation.
 - Silent target count.
 - Remaining configuration validity.
- Security
 - Authorize publication separately from activation.
 - Require additional review for fleet-wide or security-sensitive changes.
 - Store secret references and fetch secrets through their own scoped channel.
- Retry discipline
 - Retry artifact downloads by immutable hash with jitter.
 - Rate-limit acknowledgments and aggregate durable target state.
 - Retry a failed activation CAS only after re-reading the group head; a blind retry with the old expected generation is a guaranteed 409.
 - Never retry a rejected local application with the same artifact; report it and wait for a new assignment.
- Tenant isolation
 - Scope artifacts, groups, deployments, and assignments to the tenant.
 - A tenant's token receives 404 on another tenant's IDs, not 403, to avoid existence leaks.
 - Authenticate each target as a member of exactly one tenant and group, so an agent cannot read or acknowledge another group's assignment.
 - Quota artifact bytes, retained versions, and acknowledgment rate per tenant so one tenant's rollout cannot starve the shared ack pipeline.
 - Bind the signing key identifier to the tenant in the signed manifest so a valid signature for one tenant never verifies for another.

- Cell target groups so a rollout-controller stall affects one tenant's cells, not the region.
- Rollout and rollback safety
 - Test old and new binary/schema combinations before expanding the ring.
 - Canary the distribution service itself by fault domain; agents on cached config do not notice a control-plane deploy.
 - Static stability: agents keep serving the last verified assignment through any orchestrator or database outage; only new activations pause.
 - Manifest format changes follow expand, migrate, contract.
 - Old agents must ignore unknown manifest fields, and new agents must accept manifests signed by the previous orchestrator version.
 - Rolling back the orchestrator cannot lower a group's activation generation because the head only advances.
 - An orchestrator restarted mid-rollout resumes from persisted deployment phase and gate evidence, never from memory.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Polling plus push hints	Agents fetch versioned desired state	Simple recovery from missed events	Polling traffic	Baseline large fleet
Persistent stream	Server pushes assignments	Lower healthy latency	Connection and reconnect storms	Tight propagation SLO
Deployment-bundled config	Config travels with the binary	Easy compatibility boundary	Slow independent changes	Rare changes tied to code releases

- IC4 stretch — stronger requirement (immediate revocation)
 - For immediate authorization revocation, add an authoritative freshness boundary from [s2.05](#).
 - Offline cached configuration alone cannot meet that contract.
- IC4 stretch — 10× load
 - Split target groups into cells with independent rollout controllers.
 - Distribute immutable manifests through regional caches.
- IC4 stretch — cost and blast radius
 - Per-cell gates cost more orchestration but cap a bad rollout's reach.
 - Adaptive waves should honor absolute affected-request budgets, not only target percentages.
- IC4 stretch — migration
 - Publish schema support before activating fields that require it.
 - Remove old schema readers only after target inventory proves adoption.
- Weakest assumption
 - The application has a safe atomic application or rollback boundary.
 - Mitigation: make each consumer declare its application boundary (pointer swap, transaction, or irreversible) in the schema, and refuse generic rollback for the irreversible class.
- Over-engineering warning
 - Do not introduce fleet-wide consensus for every agent acknowledgment.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Scope	Separate artifacts from activation	Classify security freshness	No indefinite cached authorization
Capacity	Size polls and artifact egress	Cell and cache placement	Numbers force fan-out design

Phase	IC3 must land	IC4 adds	Separating signal
Architecture	Compiler, artifact store, fenced deployment authority, agent, ack ingestion with one trace	States the fan-out and serialization cost each split pays	Every component has a mechanism and a price
Correctness	CAS generation and owner epoch	Cohort-aware rollout authority	A stale worker cannot undo rollback
Recovery	Snapshot after watch gap	Schema migration and irreversible changes	Rollback has a declared boundary
Wrap	Recap and the application-boundary tradeoff	Names the weakest assumption before being asked	Identifies the flaw first

Interview pushback and validation

- Q1: Why is rollback a higher generation?
 - **IC3 baseline**
 - Agents must reject old decisions while still allowing previously approved bytes.
 - **IC4 stretch**
 - Use an activation log so rollback and concurrent rollout cannot overwrite each other.
- Q2: Does a download acknowledgment pass the gate?
 - **IC3 baseline**
 - No
 - Require loaded assignment and workload health.
 - **IC4 stretch**
 - Gate by client cohort and request volume to avoid hiding a small critical population.
- Q3: Two publishers choose the same next generation?
 - **IC3 baseline**
 - Only the expected-head and owner-epoch transaction may activate.
 - **IC4 stretch**
 - Store artifacts by hash so losing publishers cannot overwrite winning bytes.
- Q4: The agent was offline through every change?
 - **IC3 baseline**
 - Fetch a complete signed current assignment and verify compatibility.
 - **IC4 stretch**
 - Apply a declared freshness policy for security-sensitive configuration.
- Q5 (premise reversal): Applying this config changes a database schema, so is rollback still just a higher generation?
 - **IC3 baseline**
 - A pointer rollback is insufficient for an irreversible migration.
 - **IC4 stretch**
 - Use expand/contract migration with separate data and configuration gates.
- Q6: What if a trusted signer is compromised?
 - **IC3 baseline**
 - Pinned keys alone cannot distinguish malicious valid signatures.
 - **IC4 stretch**
 - Use emergency trust-key revocation and a separately controlled recovery channel with scoped signing authority.
- Q7: Why not push every change through a persistent stream and drop polling?
 - **IC3 baseline**
 - A stream lowers healthy latency, but a missed or compacted event still needs a versioned snapshot fetch, so polling stays as the recovery path.

- **IC4 stretch**
 - Add the stream as an accelerator with reconnect jitter and per-cell connection caps, and measure whether the tighter SLO justifies the reconnect-storm risk.
- Validation scenarios
 - Race activation and rollback from two orchestrators.
 - Replay an older signed manifest after a higher generation is applied.
 - Swap tenant or generation metadata without resigning and reject it.
 - Send duplicate, future-generation, and stale-sequence acknowledgments.
 - Fail local application after download and retain the old active instance.
 - Disconnect a critical cohort and verify the gate remains incomplete.
 - Deploy a new orchestrator version mid-rollout and verify it resumes the same deployment phase from persisted state.

Recall card

- “Artifact versions identify bytes; activation generations order decisions.”
- “A signature must bind scope as well as content.”
- “Loaded is stronger than downloaded; healthy is stronger than loaded.”
- “Rollback is a new decision, not a lower counter.”
- Wrap-up script
 - “The service stores immutable validated artifacts and activates them through a fenced group authority.”
 - “Agents verify signed assignments before an atomic local application.”
 - “Cohort-aware gates combine exact acknowledgments with workload health.”
 - “Rollback remains safe only while the application and freshness policy permit the previous configuration.”

References

- [Ranked bank](#): prompt and provenance.
- [Role framing](#): IC3 baseline, IC4 stretch, and round clock.
- [OCI role framing](#): req decode and the control-plane vocabulary that scores.
- [API/schema companion](#): related contract drills.
- Reference check: 2026-09-10.
 - Official documentation supports the mechanisms cited next to each link.
 - Capacity values and architecture choices remain proposed interview assumptions.
- [etcd API guarantees](#): revision ordering and watch limitations.
- [The Update Framework specification](#): signed metadata, version checks, and expiration as distribution defenses.